1. Introduction

In this project, my first step was to understand the Footmen vs. Archers problem and what is required to build an agent that makes decisions using Alpha-Beta search. After studying the setup and thinking about how the game evolves, I realized that the effectiveness of the agent mainly depends on two core components:

1. The evaluation function, which measures how good or bad a game state is.
2. The ordering heuristic, which determines the order actions are explored and therefore affects pruning efficiency.

With this in mind, I started identifying which features of the game state should be included in the evaluation. At first, I experimented with around 10–11 different features. Later, after observing their impact and simplifying where possible, I reduced them to 5–6 key features that capture the essential aspects without unnecessary complexity.

At the same time, I worked on designing the ordering heuristic for both footmen and archers. For the footmen, the idea was simple: if an attack is available, it should always be considered first, because attacking immediately is almost always the strongest option. After that, I prefered moves that reduce the distance to the nearest archer, since closing the gap is essential for melee units. Neutral moves where the distance stays the same, came next, and the lowest priority was given to moves that increase the distance, because these look like retreating and usually harm the agent's chances. This ordering helped the search explore the most meaningful aggressive actions first and prune many actions. For the archers, the reasoning was the opposite. I first checked whether the archer is in danger, meaning a footman can reach them soon. In that case the heuristic prefers escape actions or moves that increase distance, maintain mobility, or relocate the archer to a safer position. If the archer is not in danger, then safe attacks are preferred, followed by mobility-preserving moves and finally neutral or unnecessary repositioning. This heuristic ordering significantly increased pruning and allowed the Alpha-Beta search to cut off many branches early.

For the implementation I began coding the agent myself. I implemented the structure of the agent, the game-state model, movement and attack handling, and the general Alpha-Beta search loop. Whenever I got stuck mainly due to Java-specific details :) I used large language models (GPT/Codex) as coding assistants. My workflow was always the same: whenever I got stuck in coding I explained the logic of the function I wanted to implement and let the model propose a draft version, and then I reviewed and modified it as needed. So I used the LLM not as a direct solution generators but as a coding assitants.

By following these steps I was able to construct all the main sections of the agent: the Alpha-Beta search, the evaluation function, the action-ordering strategy, the joint-action generation and the remaining sections. Each part was developed sometimes directly, sometimes with assistance when I reached a technical difficulty.

The difficulties and the challenges that I faced while designing the alg logic and the evaluation function will be discussed in section 2 and 4.

2. Difficulties Encountered

During this assignment, I faced several difficulties across both the design and implementation stages. Some of these challenges were related to Java programming details(i already discussed this in Section 1), and others were connected to the conceptual design.and these these design-related difficulties can be separated into two main parts: 1. choosing the features for the evaluation function, and 2. selecting the correct weights to multiply with these features.

2.2 Difficulties in Designing the Evaluation Function

At the beginning I tried to include almost everything I thought might help and I ended up with 11 different features:

- ***totalDist*** → total Manhattan distance from all Footmen to all Archers. For the distence
- ***minDist*** → the minimum distance between any Footman and any Archer
- ***totalArcherHP*** → sum of all Archer hitpoints
- ***totalFootHP*** → sum of all Footman hitpoints
- ***archersAlive*** → number of Archers still alive
- ***nextTurnThreat*** → how many Footmen will be in attack range next turn
- ***adjacentNow*** → how many Footmen are directly adjacent to an Archer
- ***mobility*** → how many empty tiles the Archers can move into
- ***pressure*** → how many Footmen are within radius 2 of any Archer
- ***centroidDist*** → distance of Footmen to the "center" of the Archers
- ***spread*** → how spread out the Footmen are relative to each other(I thinked that its better for the foormen to be colse to each other)

so at the first my utility function looked like this:

utility(s) = $(w_1 \times totalDist) + (w_2 \times minDist) + (w_3 \times totalArcherHP) + (w_4 \times totalFootHP) + (w_5 \times archersAlive) + (w_6 \times nextTurnThreat) + (w_7 \times adjacentNow) + (w_8 \times mobility) + (w_9 \times pressure) + (w_{10} \times centroidDist) + (w_{11} \times spread)$

At that stage, the evaluation function was extremely noisy. Many features conflicted with each other, and the agent sometimes acted in confusing ways. So through repeated experiments and manually evaluating some example states, I discovered that many of these features were unnecessary or even harmful, so I removed some of them and these are the features I removed and why:

- ***minDist*** → confused the agent because totalDist already carries similar information
- ***nextTurnThreat*** → correlated with totalDist and adjacentNow
- ***mobility*** → useful but too noisy; Archers were sometimes encouraged to run unnecessarily
- ***pressure*** → correlated with adjacentNow, no added value
- ***centroidDist*** → inconsistent and not reliable
- ***spread*** → did not meaningfully improve decisions

After simplification, the evaluation function became much more stable and predictable.

Another thing that I want to poin at is that during the feature engineering stage, I also experimented with different distance metrics. I tried *Manhattan distance*, *Euclidean distance*, and even *Cosine distance*.

- $\text{ManhattanDistance}(u, v) = |x_1 - x_2| + |y_1 - y_2|$

- $\text{EuclideanDistance}(u, v) = \sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2}$
- $\text{CosineDistance}(u, v) = 1 - \frac{\vec{u} \cdot \vec{v}}{|\vec{u}|\,|\vec{v}|}$

But after thinking about the movement rules in the environment, I felt that Manhattan distance was already enough and more suitable for this assignment, since units cannot move diagonally and movement naturally follows a grid-like structure.

2.3 Weight Tuning and Balancing Difficulties

Choosing the weights for each feature was also a challenge. Initially, I assigned weights purely based on intuition. However, after testing many examples manually and running simulations, I adjusted the weights several times to find a more balanced combination. During this process, I realized that the agent's behavior is extremely sensitive to weight changes. This made me think that using metaheuristics (such as Genetic Algorithms, Simulated Annealing, or Particle Swarm Optimization) to automatically tune the weights could significantly improve performance. But since this was outside the scope of the assignment, I focused on manual tuning :).

3. Prompt Design and Usage

During the assignment, I used prompts mainly when I faced a difficulty while coding or when I needed to translate my ideas into clean and correct Java code. My general workflow like a explaiined before was simple: whenever I got stuck I explained to the LLM exactly what I was trying to implement, the logic behind it, and the structure the function should have and After the model generated a solution, I carefully read the output, corrected it when needed, and adapted it to my agent.

Sometimes I also used a small trick 😉 to get better results. Instead of directly asking for the code, I first opened a separate chat and explained all my thoughts, reasoning, and design choices in a rough way, almost like talking to myself. After that, I asked the LLM to restructure those thoughts and turn them into a clean and well-organized prompt. Then I copied that refined prompt into a new chat and used it to generate clearer, more accurate code. This helped a lot, especially when implementing complicated parts like collision handling, movement ordering, and sections of the evaluation function.

4. Development Tools and LLMs

Development Tools:

- VS Code (main IDE) & Eclipse IDE
- Java 8
- SEPIA environment & provided libraries

Large Language Models Used:

- GPT-5.1
- GPT-5.1-codex-max

5. Errors and Solutions

During the assignment, the main issue I encountered was caused by the *minDist* feature in the evaluation function. In Scenario 1 (2FvA), this feature confused the agent and created an unexpected loop in the behavior. The two footmen kept moving left and right repeatedly, never committing to an attack, and eventually died. After several experiments, I realized that *minDist was giving misleading signals* and dominating certain action selections. Removing this feature completely solved the problem and immediately stabilized the agent's behavior. Other than this issue, I did not face major coding or runtime errors. Most small Java-related issues were resolved through simple debugging and reviewing the code and sometimes by asking an LLM for clarification when I was unsure about implementation details.